Git 2.56 · GitHub

Git en equipo, sin pánico

De pisarse el código a ramas, pull requests y conflictos que resuelves con calma.



Chamba Lab · 2026



Git en equipo

Cada tarea en su rama; a main solo entra lo revisado.

- Trabajas en paralelo sin pisar a nadie
- El pull request muestra qué cambió y por qué
- Si dos tocan la misma línea, Git avisa: no adivina

Anatomía de una rama: un nombre que apunta a tu último commit

`feat` / `42` - `pago-yape`

tipo issue qué hace

Tipo

Qué clase de cambio es: `feat` agrega, `fix` corrige.

Issue

El número de la tarea en GitHub: la #42 es «Aceptar Yape».

Qué hace

Dos o tres palabras, en minúscula y con guiones.

Tipos de rama: convenciones del equipo, no reglas de Git

Principal

Vive siempre y está protegida: lo que hay ahí funciona.

`main`

TODOS LOS REPOS

De tarea

Una por issue; dura horas o días y se borra al fusionar.

`feat/42-pago-yape`

`fix/51-total`

EL DÍA A DÍA

Urgente

Sale de main para corregir algo que ya está en producción.

`hotfix/igv`

DE VEZ EN CUANDO

De versión

Git Flow: equipos que publican versiones con fecha.

`develop`

`release/2.0`

SOLO ALGUNOS EQUIPOS

Quién participa: cuatro piezas alrededor de main

Tu repo local
`git commit`

GitHub (origin)
`git push · git pull`

main
protegida

Revisores
`Approve · Request changes`

Checks (CI)
`tests en GitHub Actions`

A main solo entra lo que pasó por un pull request, una revisión y checks en verde.

Rama por tarea: sale de main y vive aparte



- Nace del main recién actualizado
- Tus commits viven ahí: main no se entera
- `push -u` la sube a GitHub y la vincula

```
git switch main
git pull
git switch -c feat/42-pago-yape
# editas src/pagos.js
git add src/pagos.js
git commit -m "feat: aceptar Yape"
git push -u origin feat/42-pago-yape
```

Crea la rama y se cambia a ella.

Commits que se entienden: uno por idea, con su tipo

git log --oneline

```
# así sí: cada línea cuenta algo
9c1e2f4 test(pagos): cubrir pago con Yape
4b7a0d2 feat: aceptar Yape
e81c3a9 docs: listar métodos de pago

# así no: nadie sabe qué pasó
f00d123 cambios
ab12cd3 ahora sí funciona
77aa001 arreglos varios
```

tipo(alcance): qué hace, en imperativo y sin punto final. El alcance es opcional.

`feat` y `fix` los define Conventional Commits; `docs`, `test` o `chore` son ejemplos usuales.

Pequeños: si el mensaje necesita un «y», eran dos commits.

El pull request: pides que tu rama entre a main



Los gatos del otro piso revisan el diff, no tus intenciones

```
gh pr create \
  --base main \
  --title "feat: aceptar Yape" \
  --body "Closes #42" \
  --reviewer bodega/gatos
```

Al fusionar en main, cierra solo el issue #42.

- Lo mismo se hace con el botón de GitHub
- Pequeño: se revisa en minutos, no en días
- ¿No está listo? Ábrelo como borrador con `--draft`

La revisión: tres respuestas posibles

Comment

Opina sin aprobar ni bloquear.

`¿y si el monto es 0?`

DUDAS E IDEAS

Approve

Los cambios están listos para fusionarse.

`LGTM`

LO QUE BUSCAS

Request changes

Hay algo que corregir antes de fusionar.

`falta el test`

NO ES PERSONAL

Si te piden cambios

Nuevo commit en la misma rama: el PR se actualiza solo.

`git push`

SIN ABRIR OTRO PR

Traer lo nuevo de main: merge o rebase

Merge




```
git fetch origin
git merge origin/main
```

Une las dos historias con un commit de merge. No reescribe nada.

Rebase




```
git fetch origin
git rebase origin/main
```

Reaplica tus commits encima de main: historia en línea, commits nuevos.

Rebase sin miedo: continuar, abortar y subir

feat/42-pago-yape

```
git fetch origin
git rebase origin/main
# ¿conflicto? resuélvelo y sigue:
git add src/pagos.js
git rebase --continue
# ¿te perdiste? vuelve a como estabas:
git rebase --abort
# tu rama ya estaba en GitHub:
git push --force-with-lease
```

Regla de oro (Pro Git): no hagas rebase de commits que otros ya usan.

En un rebase los lados se invierten: «Current» es main y «Incoming» es tu commit.

`--force-with-lease` falla si alguien subió algo que no has visto; `--force` lo pisa.



¿Por qué Git no resuelve el conflicto solo?

Rakim agregó Yape y el zorro, Plin. En la misma línea.

El conflicto: dos cambios en el mismo lugar



- Líneas distintas: Git los junta solo
- Misma línea: decide una persona

```
// src/pagos.js
export const METODOS_PAGO = [
  "tarjeta",
  "efectivo",
<<<<<<< HEAD
  "yape",
=======
  "plin",
>>>>>>> origin/main
];
```

Arriba, tu rama (HEAD); abajo, lo de main.

El clic del pánico: «Accept Current Change» por reflejo



Elegir un lado borra el otro: el Plin del zorro desaparece y tu PR lo saca de main.

```
// src/pagos.js, después del clic
export const METODOS_PAGO = [
  "tarjeta",
  "efectivo",
  "yape",
];

// el diff del PR decía «- "plin"»,
// pero entre 300 líneas nadie lo vio.
// Lo notó un cliente que quiso pagar
// con Plin.
```

Resolver paso a paso: junta los dos lados y prueba



¿No entiendes el otro lado? Resuélvelo en pareja con su autor

```
git status
# both modified: src/pagos.js
# edita: deja "yape" y "plin"
# y borra <<<<<<< ======= >>>>>>>
git add src/pagos.js
git commit   # cierra el merge
npm test     # ¿todo sigue bien?
git push
# ¿perdido? git merge --abort
```

`git add`: este archivo ya está resuelto.

Matriz de decisión: merge o rebase

	Merge	Rebase
Historial	El real, con commits de merge	En línea, como si fuera en serie
¿Reescribe commits?	No	Sí: push con `--force-with-lease`
Rama que otros usan	Seguro	No: rompe su trabajo
Botón del PR en GitHub	Create a merge commit	Rebase and merge

Rama solo tuya: rebase sobre `origin/main` antes del PR. Rama compartida: merge. Al fusionar, el botón que fije tu equipo.

Lo que te llevas

- Una rama por tarea, que nace de main actualizado
- PR pequeño, con `Closes #42` y revisión antes de main
- Un conflicto se lee: junta los dos lados y prueba

```
> libro: git-scm.com/book/es/v2
> docs: git-scm.com/docs/git-merge
> GitHub: docs.github.com/es/pull-requests
> repo: github.com/Chamba-Lab
```